

MACHINE LEARNING ENGINEERING

FASE 4 | AULA 04 -

ANÁLISE DE SENTIMENTO

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	17
REFERÊNCIAS	18

O QUE VEM POR AÍ?

Uma tarefa comum em Processamento de Linguagem Natural é a Análise de Sentimento. Suas aplicações são variadas e em diversas áreas, como marketing, atendimento ao cliente, pesquisa de mercado etc.

Ao conseguir identificar quais comentários são bons ou ruins para a sua empresa ou produto, podemos filtrar e entender mais a fundo maneiras de melhorar, o que pode ser muito importante e poderoso para a empresa.

HANDS ON

Como monitorar a opinião dos clientes a respeito de um produto? Ou da empresa? Ou de um político? Esse é um desafio da análise de sentimento, uma tarefa bastante comum em processamento de linguagem natural.

Nessa aula abordaremos como analisar sentimentos de diversas formas, passando por abordagens mais tradicionais até a utilização de transferência de conhecimento.

EMENDAS

SAIBA MAIS

A tarefa de análise de sentimento é uma tarefa de classificação comum em Processamento de Linguagem Natural.

Você pode desenvolver um modelo de análise de sentimento de diversas formas: treiná-lo para prever um sentimento positivo, negativo ou neutro, ou ir além e prever tristeza, raiva, decepção, medo, surpresa, felicidade, satisfação e outras variedades de sentimentos.

Além disso, você pode experimentar diversas arquiteturas e pré-processamentos, desde as mais tradicionais e simples como um pipeline que pré-processe os dados, vetorize eles usando bag-of-words e depois classifique os sentimentos usando Regressão Logística ou Árvore de Decisão, como modelos mais complexos, que usam embedding contextuais e transferência de conhecimento.

Nos vídeos, mostramos várias formas de desenvolver os modelos e os comparamos. Aqui, vamos explicar mais sobre como usar o BERT para treinar um modelo de classificação.

Há essencialmente duas formas de treinar um modelo de classificação usando o BERT:

- **Extração de features:**

Podemos lançar mão das relações aprendidas entre as palavras e seus contextos e usá-los como variáveis. Para isso usamos as camadas escondidas (hidden states) e treinamos o classificador nelas, sem modificar o modelo pré-treinado.

Congelamos os pesos do corpo durante o treinamento e usamos as camadas escondidas como variáveis para o classificador. Esse é o método mais simples e rápido, ideal caso você não tenha acesso a GPU, já que as camadas escondidas só precisam ser pré-computadas uma vez.

Vamos para um exemplo? Utilizaremos o Distilbert, mas se quiser usar o BERT basta mudar a variável `model_checkpoint`. Como estamos usando um modelo treinado em inglês, vamos seguir com um texto em inglês: "Welcome to Natural Language Processing class."

Vamos usar também a classe `AutoModel` que converte os tokens em embeddings e alimenta o componente de encoder, retornando as camadas escondidas (hidden state).

```
from transformers import AutoModel, AutoTokenizer
import torch
model_checkpoint = "distilbert-base-uncased"
device = torch.device("cuda" if torch.cuda.is_available()
else "cpu")
model = AutoModel.from_pretrained(model_checkpoint).to(device)
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
```

A primeira coisa que precisamos fazer é converter o texto para tensores Pytorch, o que pode ser feito usando o parâmetro `return_tensors="pt"` do tokenizer.

```
text = "Welcome to Natural Language Processing class."
inputs = tokenizer(text, return_tensors="pt")
print(f"Formato do input tensor: {inputs['input_ids'].size()}")
```

Saída:

```
Formato do input tensor: torch.Size([1, 9])
```

Como podemos ver, nosso tensor tem formato de 1 batch com 9 tokens. Agora, vamos colocar nosso tensor no mesmo device do modelo e usá-lo como input.

```
inputs = {k:v.to(device) for k, v in inputs.items()}
with torch.no_grad():
    outputs = model(**inputs)
print(outputs)
print(outputs.last_hidden_state.size())
```

Saída:

```
BaseModelOutput(last_hidden_state=tensor([[[[-0.4848, -0.2158, -0.1232, ..., -0.1265, 0.2086, 0.3633],
[-0.2069, -0.3256, 0.3101, ..., 0.1496, 0.1097, 0.3034],
[-0.6798, -0.2180, -0.0089, ..., -0.1475, 0.4134, 0.2263],
...,
```

```
[ 0.1489, -0.3452, 0.0863, ..., -0.2668, -0.1269,
-0.0873],
[-0.8445, -0.7973, -0.0104, ..., 0.2493, 0.1418,
-0.2717],
[ 0.8880, 0.1216, -0.5799, ..., 0.1695, -0.6401,
-0.0926]]]), hidden_states=None, attentions=None)
torch.Size([1, 9, 768])
```

O gerenciador de contexto `torch.no_grad` foi usado para desabilitar o cálculo automático do gradiente, o que nos ajuda com uma redução de memória necessária. Dependendo da configuração do modelo, o resultado pode conter vários objetos, como camadas escondidas, perdas, camadas de atenção e mais, organizadas em tuplas nomeadas. No nosso exemplo, temos somente a camada escondida.

Podemos ver agora que o vetor resultante tem formato 1 batch, 9 tokens com dimensão 768. Dessa forma, temos um vetor com dimensão 768 para cada um dos nossos 9 tokens.

Agora podemos expandir nosso código para, em vez de extrair a última camada escondida de um único texto, extrair de um dataset inteiro.

```
def extract_hidden_states(batch):
    inputs = {k: v.to(device) for k,v in batch.items() if k in
tokenizer.model_input_names}
    with torch.no_grad():
        last_hidden_state = model(**inputs).last_hidden_state
    return {"hidden_state":
last_hidden_state[:,0].cpu().numpy() }
```

Vamos testar usando um dataset armazenado no Hugging Face. Esse hub contém muito mais do que modelos, mas datasets, aplicativos e muito mais. Vamos usar o dataset [emotion](#). Ele contém mensagens de twitter em inglês classificadas em seis emoções diferentes. Para isso, primeiro precisamos instalar a biblioteca datasets rodando o comando `pip install datasets`.

```
from datasets import load_dataset
emotions = load_dataset("emotion", trust_remote_code=True)
emotions
```

Saída:

```
DatasetDict({
```

```
train: Dataset({
  features: ['text', 'label'],
  num_rows: 16000
})
validation: Dataset({
  features: ['text', 'label'],
  num_rows: 2000
})
test: Dataset({
  features: ['text', 'label'],
  num_rows: 2000
})
})
```

Aqui podemos ver que temos uma dataset de classe DatasetDict com treino (16000 registros), validação e teste separados (2000 registros cada). As colunas são text e label: a coluna text vai ser nossa variável de entrada para o treino e a coluna label é nosso objetivo, que contém o sentimento para nossa classificação supervisionada.

Para uma análise exploratória, você pode facilmente converter seus dados em um pandas dataframe:

```
import pandas as pd
emotions.set_format(type="pandas")
df = emotions["train"][:]
df.head()
```

Saída:

	text	label
0	i didnt feel humiliated	0
1	i can go from feeling so hopeless to so damned...	0
2	im grabbing a minute to post i feel greedy wrong	3
3	i am ever feeling nostalgic about the fireplac...	2
4	i am feeling grouchy	3

Como podemos ver, os labels vieram como número inteiros, mas podemos convertê-los usando o método int2str:

```
def label_int2str(row):
    return emotions["train"].features["label"].int2str(row)
df["label_name"] = df["label"].apply(label_int2str)
df.head()
```


Saída:

	text	label	label_name
0	i didnt feel humiliated	0	sadness
1	i can go from feeling so hopeless to so damned...	0	sadness
2	im grabbing a minute to post i feel greedy wrong	3	anger
3	i am ever feeling nostalgic about the fireplac...	2	love
4	i am feeling grouchy	3	anger

Agora podemos analisar o dataset. É possível notar que temos um problema de classificação não balanceado, em que "joy" e "sadness" ocorrem muito mais vezes do que "surprise", por exemplo.

```
import matplotlib.pyplot as plt
df["label_name"].value_counts(ascending=True,
normalize=True).plot.barh()
plt.title("% de frequência por classe")
plt.show()
```

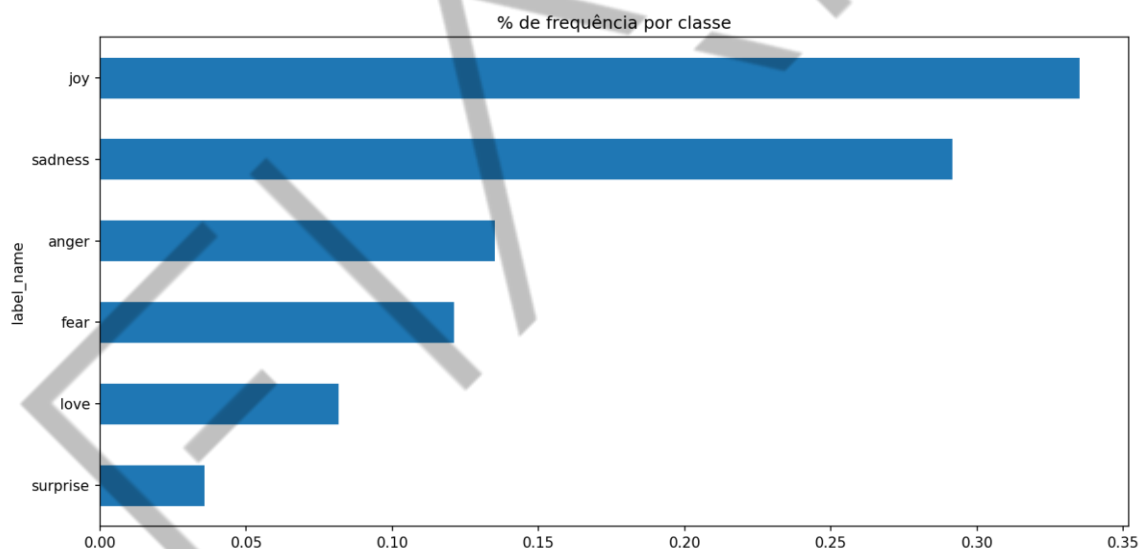


Figura 1 - Percentual de frequência por classe
Fonte: Elaborado pelo autor (2024)

Explore bem seus dados, cheque a distribuição de tamanho do texto, observe palavras que aparecem com mais frequência e tente identificar onde seu modelo pode ter alguma falha, entre outros.

Entender o tamanho máximo de um texto é bem importante, visto que nossos modelos geralmente têm um máximo que podemos achar através do `tokenizer.model_max_length`. No nosso caso é 512 e, acima disso, o texto será

truncado, ou seja, quebrado até chegar nos 512. A seguir mostramos como tokenizar um dataset. Precisamos que todos os registros tenham o mesmo tamanho; dessa forma, o padding=True atribui zeros até o tamanho do texto mais longo do lote.

```
emotions.set_format(type="torch", columns=["text",
"label"])
def tokenize(batch):
    return tokenizer(batch["text"], padding=True,
truncation=True)
print(tokenize(emotions["train"][:2]))
```

Saída:

```
{'input_ids': [[101, 1045, 2134, 2102, 2514, 26608, 102, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], [101, 1045, 2064,
2175, 2013, 3110, 2061, 20625, 2000, 2061, 9636, 17772, 2074,
2013, 2108, 2105, 2619, 2040, 14977, 1998, 2003, 8300, 102]],
'attention_mask': [[1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0], [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1]]}
```

Podemos ver que o primeiro registro é menor do que o segundo e vários zeros foram aplicados no input_ids para chegar ao mesmo tamanho. Os zeros têm um token especial [PAD] no vocabulário, que permite identificá-los e atribuir uma máscara de atenção zero também para que sejam ignorados. Outros tokens especiais são: [UNK] quando o token é desconhecido, [CLS] indica início de sentença e [SEP] uma separação, como pontuação.

Agora para aplicar a todos os nossos datasets, podemos usar o método map:

```
emotions_encoded = emotions.map(tokenize, batched=True,
batch_size=None)
```

O batch_size parâmetro igual a None faz com que a função tokenize seja aplicada no dataset inteiro de uma vez só, o que garante que os tensores de entrada e máscaras de atenção tenham o mesmo formato globalmente. O método map adiciona as colunas dentro do dataset, como podemos ver a seguir.

```
print(emotions_encoded["train"].column_names)
emotions_encoded.set_format("torch", columns=["input_ids",
"attention_mask", "label"])
```

Saída:

```
['text', 'label', 'input_ids', 'attention_mask']
```

Agora podemos extrair as camadas escondidas aplicando a nossa função:

```
emotions_hidden = emotions_encoded.map(extract_hidden_states, batched=True)
import numpy as np
X_train = np.array(emotions_hidden["train"]["hidden_state"])
X_validation = np.array(emotions_hidden["validation"]["hidden_state"])
y_train = np.array(emotions_hidden["train"]["label"])
y_validation = np.array(emotions_hidden["validation"]["label"])
```

Agora é só aplicar o classificador da sua escolha.

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression(max_iter=3000)
lr.fit(X_train, y_train)
lr.score(X_validation, y_validation)
```

Saída:

```
0.634
```

Agora pode-se iterar esse modelo inicial para melhorá-lo usando técnicas de otimização de hiperparâmetros, balanceamento de classes ou testar outros modelos.

- **Fine-tuning:**

Um modelo novo é treinado do início ao fim, atualizando também os parâmetros do modelo pré-treinado.

Conforme falamos na aula passada, o BERT é o corpo do modelo, mas para usá-lo para a tarefa de classificação de análise de sentimento precisamos treiná-lo adicionando uma cabeça. Essa cabeça precisa ser diferenciável e por isso normalmente o classificador aplicado é uma rede neural.

Esse método faz com que retreinemos as camadas escondidas, que servirão de variável de entrada para o modelo. Isso é especialmente importante quando os dados utilizados no pré-treino do modelo não são adequados ao seu problema de classificação específico.

Prontos(as) para conhecer a Trainer API do hugging face?

O primeiro passo é baixar o modelo pré-treinado, assim como no método anterior. No entanto, em vez de usarmos a classe `AutoModel`, vamos usar a classe `AutoModelForSequenceClassification`, que já possui uma cabeça de classificação em cima do corpo do modelo pré-treinado e pode ser facilmente treinada com o modelo base.

Além disso, precisamos definir a quantidade de labels que o modelo deverá prever, que no nosso caso são 6 sentimentos. Isso é importante porque define o número de resultados na cabeça de classificação.

```
from transformers import AutoModelForSequenceClassification
num_labels = 6
model = AutoModelForSequenceClassification.from_pretrained(
    model_checkpoint, num_labels=num_labels
).to(device)
```

O próximo passo é definir as métricas de performance que serão utilizadas para avaliar a performance do modelo durante o fine-tuning. Para isso, criaremos uma função `compute_metrics()` que usaremos dentro do Trainer. Ela receberá um objeto `EvalPrediction`, que é uma tupla nomeada com atributos `predictions` e `label_ids`, e retornará um dicionário com os nomes das métricas e seus valores.

Como vimos anteriormente, nosso problema é de classificação desbalanceada; então, nesse exemplo usaremos a métrica F1-score. Também adicionaremos acuracidade para referência. Aqui você pode escolher a melhor métrica para seu problema.

```
from typing import Dict
from sklearn.metrics import accuracy_score, f1_score
def compute_metrics(pred) -> Dict[str, float]:
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)
    return {
```

```
"f1": f1_score(labels, preds, average="weighted"),  
"accuracy": accuracy_score(labels, preds)  
}
```

Agora vamos definir os hiperparâmetros para o treinamento usando a classe `TrainingArguments`. O parâmetro mais importante a ser especificado é o `output_dir`, que armazenará todos os artefatos do treinamento.

```
from transformers import TrainingArguments  
batch_size = 64  
logging_steps = len(emotions_encoded["train"]) //  
batch_size  
model_name = f"{model_checkpoint}-finetuned-emotion"  
training_args = TrainingArguments(  
    output_dir=model_name,  
    num_train_epochs=2,  
    learning_rate=2e-5,  
    per_device_train_batch_size=batch_size,  
    per_device_eval_batch_size=batch_size,  
    weight_decay=0.01,  
    eval_strategy="epoch",  
    disable_tqdm=False,  
    logging_steps=logging_steps,  
    log_level="error"  
)
```

Caso você queira salvar seu modelo dentro do hugging face hub, você precisa se autenticar usando o comando a seguir e adicionar aos parâmetros acima o `push_to_hub=True`. Treinaremos o nosso modelo no mesmo dataset que usamos e transformamos no método de extração de variáveis.

```
from hugging face import notebook_login  
notebook_login()
```

Agora é só treinar. Lembrando que esse treinamento é bem mais complexo; logo, demora mais e utiliza mais recursos.

```
from transformers import Trainer  
trainer = Trainer(  
    model=model,  
    args=training_args,  
    compute_metrics=compute_metrics,  
    train_dataset=emotions_encoded["train"],  
    eval_dataset=emotions_encoded["validation"],  
    tokenizer=tokenizer
```

```
)  
trainer.train()
```

Saída:

```
{'loss': 0.8256, 'grad_norm': 4.6312642097473145,  
'learning_rate': 1e-05, 'epoch': 1.0}  
{'eval_loss': 0.3153814673423767, 'eval_f1': 0.9088253579527027,  
'eval_accuracy': 0.91, 'eval_runtime': 59.1891,  
'eval_samples_per_second': 33.79, 'eval_steps_per_second': 0.541,  
'epoch': 1.0}  
{'loss': 0.2494, 'grad_norm': 5.666621685028076,  
'learning_rate': 0.0, 'epoch': 2.0}  
{'eval_loss': 0.21286585927009583, 'eval_f1':  
0.9299820097365558, 'eval_accuracy': 0.93, 'eval_runtime': 74.5999,  
'eval_samples_per_second': 26.81, 'eval_steps_per_second': 0.429,  
'epoch': 2.0}  
{'train_runtime': 5481.6453, 'train_samples_per_second': 5.838,  
'train_steps_per_second': 0.091, 'train_loss': 0.5375191497802735,  
'epoch': 2.0}  
TrainOutput(global_step=500, training_loss=0.5375191497802735,  
metrics={'train_runtime': 5481.6453, 'train_samples_per_second':  
5.838, 'train_steps_per_second': 0.091, 'total_flos':  
720342861696000.0, 'train_loss': 0.5375191497802735, 'epoch': 2.0})
```

Podemos ver que nosso modelo teve uma performance muito melhor do que com a abordagem de extração de variáveis anterior. Chegamos em uma acuracidade de 93%!

Para realizar inferências e desbravar a performance do modelo, você pode simplesmente usar o método predict.

```
import numpy as np  
preds_output =  
trainer.predict(emotions_encoded["validation"])  
print(preds_output.metrics)  
y_preds = np.argmax(preds_output.predictions, axis=1)
```

Saída:

```
{'test_loss': 0.21286585927009583, 'test_f1':  
0.9299820097365558, 'test_accuracy': 0.93, 'test_runtime':  
52.7476, 'test_samples_per_second': 37.916,  
'test_steps_per_second': 0.607}
```

Vemos que o método `predict` também resulta em um objeto com atributo `metrics` que contém as métricas daquele lote. Além disso, com o `y_preds` já podemos calcular uma matriz de confusão comparando com o `y_valid` e entendermos cada vez mais o comportamento do nosso modelo.

Caso você não tenha usado o `push_to_hub` nos argumentos mas queira salvar o modelo no hub após o treinamento, isso também é possível rodando o comando `trainer.push_to_hub(commit_message="training completed")`. Com isso, você pode usar seu modelo para novas inferências e compartilhá-lo. Para utilizá-lo depois de mandar para o hub, basta usar com o pipeline:

```
from transformers import pipeline
model_id = f"<HUB-USERNAME>/{model_name}"
classifier = pipeline("text-classification",
model=model_id)
novo_tweet = "I love BERT and NLP"
preds = classifier(novo_tweet, return_all_scores=True)
```

No fine-tuning, um modelo estado-da-arte pré-treinado geralmente é um bom caminho, a não ser que você tenha poucos dados classificados para o treinamento; nesse caso uma extração de features seria mais adequada.

Desafios e Considerações

- **Ambiguidade e Sarcasmo**

Se houver muito sarcasmo dentro de nossos dados de treino, o modelo também pode criar padrões não verdadeiros, generalizando erroneamente os sentimentos.

Além disso, na falta de "labels naturais" para identificar a emoção de um texto, muitas vezes criamos regras para gerá-los, as quais funcionam para a maioria dos casos. No entanto, outros podem não fazer sentido.

Um exemplo é anotar positivo para tweets com :), que podem muitas vezes expressar um sentimento positivo, mas sarcasmo também. O modelo vai sempre aprender padrões entre os dados e o label; dessa forma, ter labels consistentes e corretos é essencial.

- **Multilinguismo**

Temos desafios de análise de sentimento em diferentes idiomas e a necessidade de modelos específicos para cada um deles. Como vimos, a maioria dos

modelos que temos disponíveis são focados em resolver problemas em inglês e são poucos os que estão adaptados para o português, como o BERTimbau.

- **Viés e Ética**

Todos os modelos, assim como as pessoas, possuem vieses. O BERT, por exemplo, mesmo sendo treinado em dados considerados neutros, possui viés de gênero.

Sua própria documentação indica isso na seção de [limitações e vieses](#), em que, quando se requisita profissões de homens, os resultados são: carpinteiro, garçom, barbeiro, mecânico e vendedor, enquanto para mulheres os resultados são: enfermeira, garçonete, babá, prostituta e cozinheira.

Dessa maneira, precisamos ficar atentos e atentas às limitações e vieses dos nossos modelos para evitar causar danos à sociedade.

O QUE VOCÊ VIU NESTA AULA?

Nessa aula você aprendeu como desenvolver modelos para analisar sentimentos, usando desde métodos tradicionais até modelos pré-treinados como o BERT, tanto para a extração de variáveis como para o fine-tuning, comparando seus resultados.

EMENDAS

REFERÊNCIAS

HUGGING FACE. **bert-base-uncased**. 2024. Disponível em: <<https://huggingface.co/google-bert/bert-base-uncased>>. Acesso em: 31 jul. 2024.

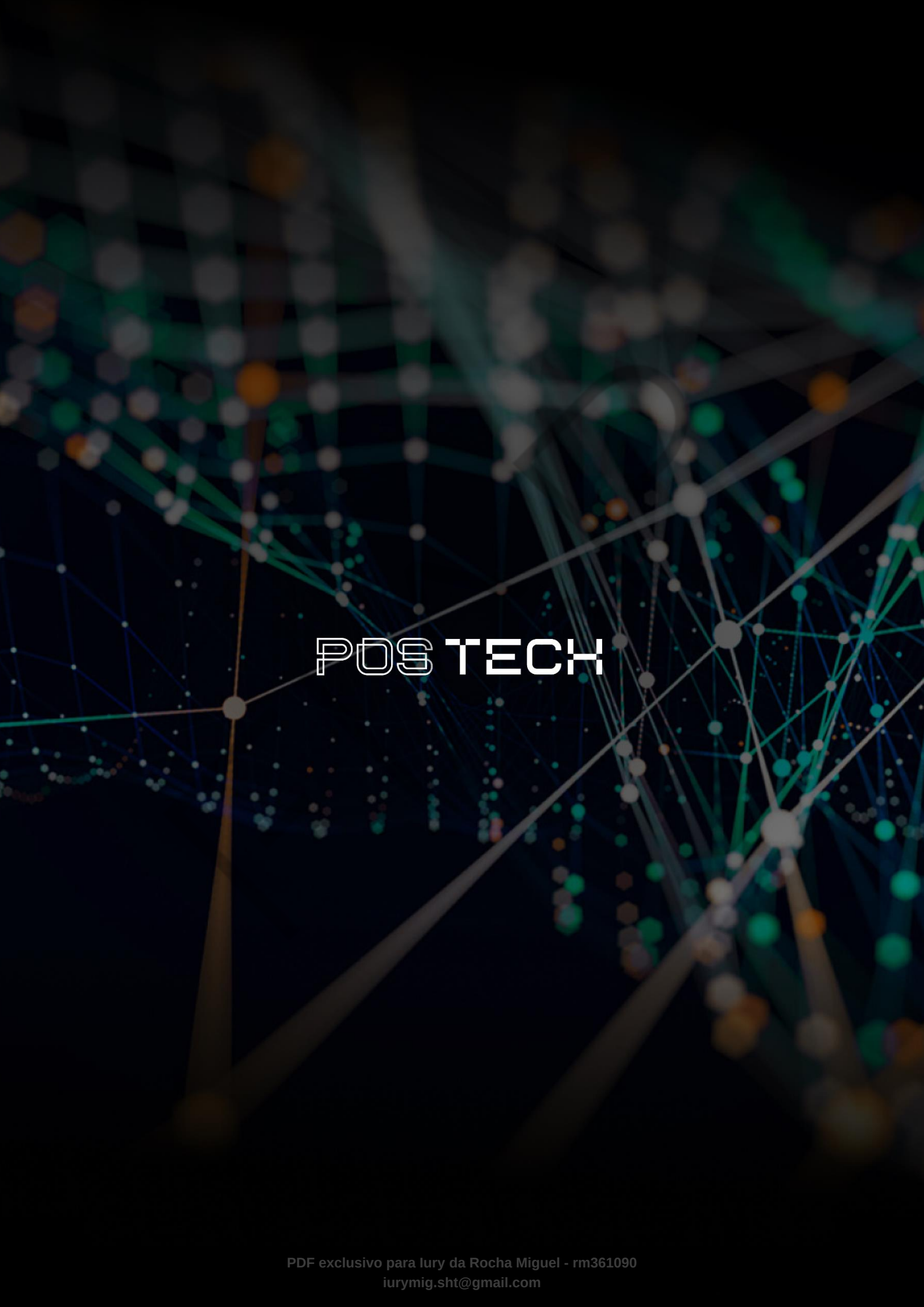
HUGGING FACE. **dair-ai/emotion**. 2024. Disponível em: <<https://huggingface.co/datasets/dair-ai/emotion>>. Acesso em: 31 jul. 2024.

TUNSTALL, L.; VON WERRA, L.; WOLF, T. **Natural Language Processing with Transformers**. Revised Edition. [s.l.]: O'Reilly Media, Inc., 2022.

PALAVRAS-CHAVE

Processamento de Linguagem Natural. Transformers. Transferência de Conhecimento. BERT. AI Generativa. Aprendizado de Máquina. Análise de Sentimento.

EMENDAS



POSTECH